

MSc Robotics

Software Development for Robotics

Practical Assignment 6

Gonçalo Pinheiro (s3741680)
Ceyhun Ceylan (s3514609)

m.r.q.p.goncalo@student.utwente.nl
c.ceylan@student.utwente.nl

April 2026

Faculty of Electrical Engineering, Mathematics and Computer Science
University of Twente

1 Introduction

This report describes the development of the control software for Assignment 6 of Software Development for Robotics. The assignment focuses on building a complete control chain for the RELbot, starting in simulation and moving towards execution on the physical robot. A central part of this process is the construction of a vision-based closed-loop controller that can detect a green object and steer the RELbot towards it.

The final setup contains the following main parts:

- a camera input node that receives webcam frames over UDP,
- a detection node that estimates the position of the green object,
- a sequence controller that computes wheel commands,
- the RELbot adapter and simulator, and
- a turtlesim branch for visualisation.

To keep the report easy to complete, figure placeholders are already included. If the referenced image file exists, LaTeX will include it automatically. Otherwise, a visible placeholder box is shown in the document.

Figure ?? shows the software interface scheme of the implemented simulation.

2 Background: RELbot Controller Architecture

The RELbot can be viewed as a Cyber-Physical System (CPS) that is divided into three timing domains:

- the **Soft Real-Time (SRT)** layer, implemented in ROS 2,
- the **Firm Real-Time (FRT)** layer, implemented using the XRF2 framework, and
- the **Hard Real-Time (HRT)** physical layer consisting of the robot hardware, sensors, and actuators.

For this report, the main work happens in the SRT layer. This is where the image is processed and where the movement commands are generated. The FRT and HRT layers are not yet used directly in Sections 6.1–6.3, but they are still relevant because the simulation software is meant to be moved to the real RELbot later.

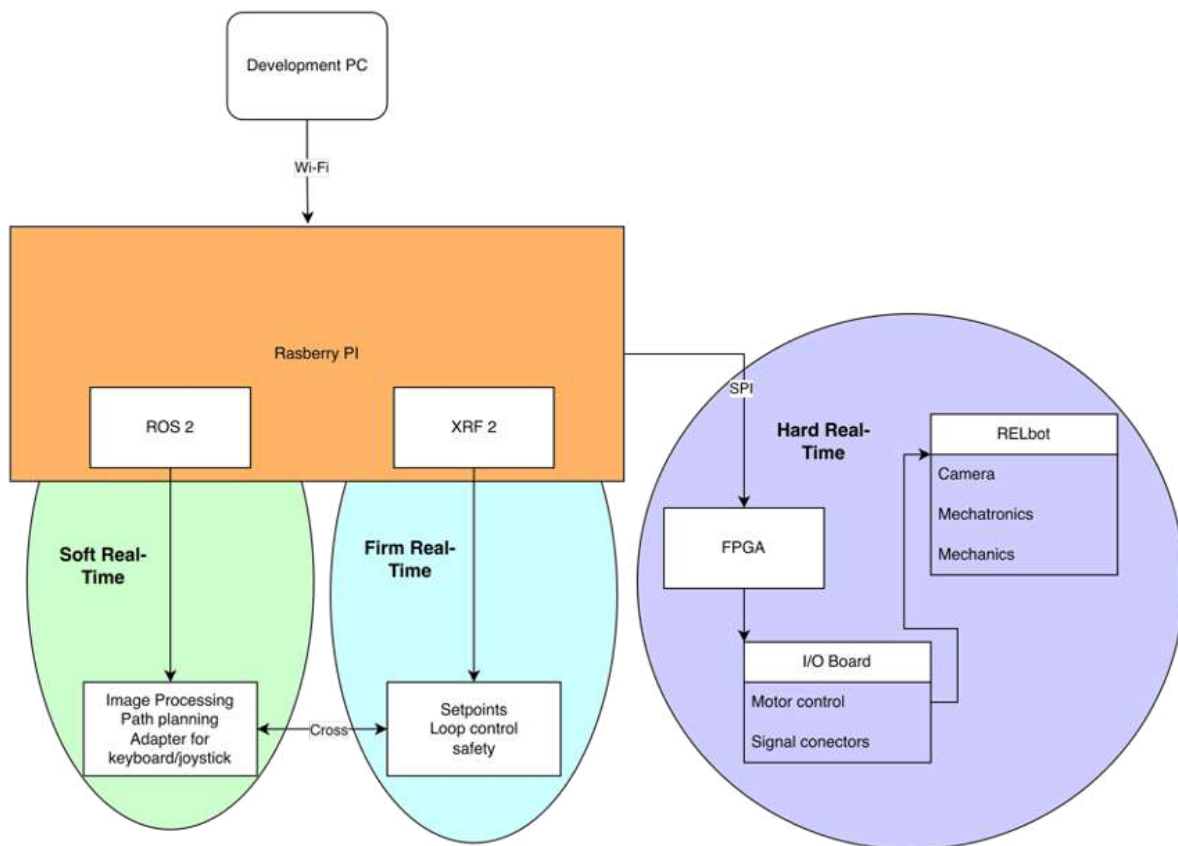


Figure 1: Cyber-Physical System

Cyber-Physical System view of the RELbot architecture. This figure shows the separation between the soft real-time ROS 2 layer, the firm real-time XRF2 layer, and the hard real-time hardware domain.

2.1 Relevant software blocks

Only the blocks that are relevant to the current Assignment 6 scope are discussed here.

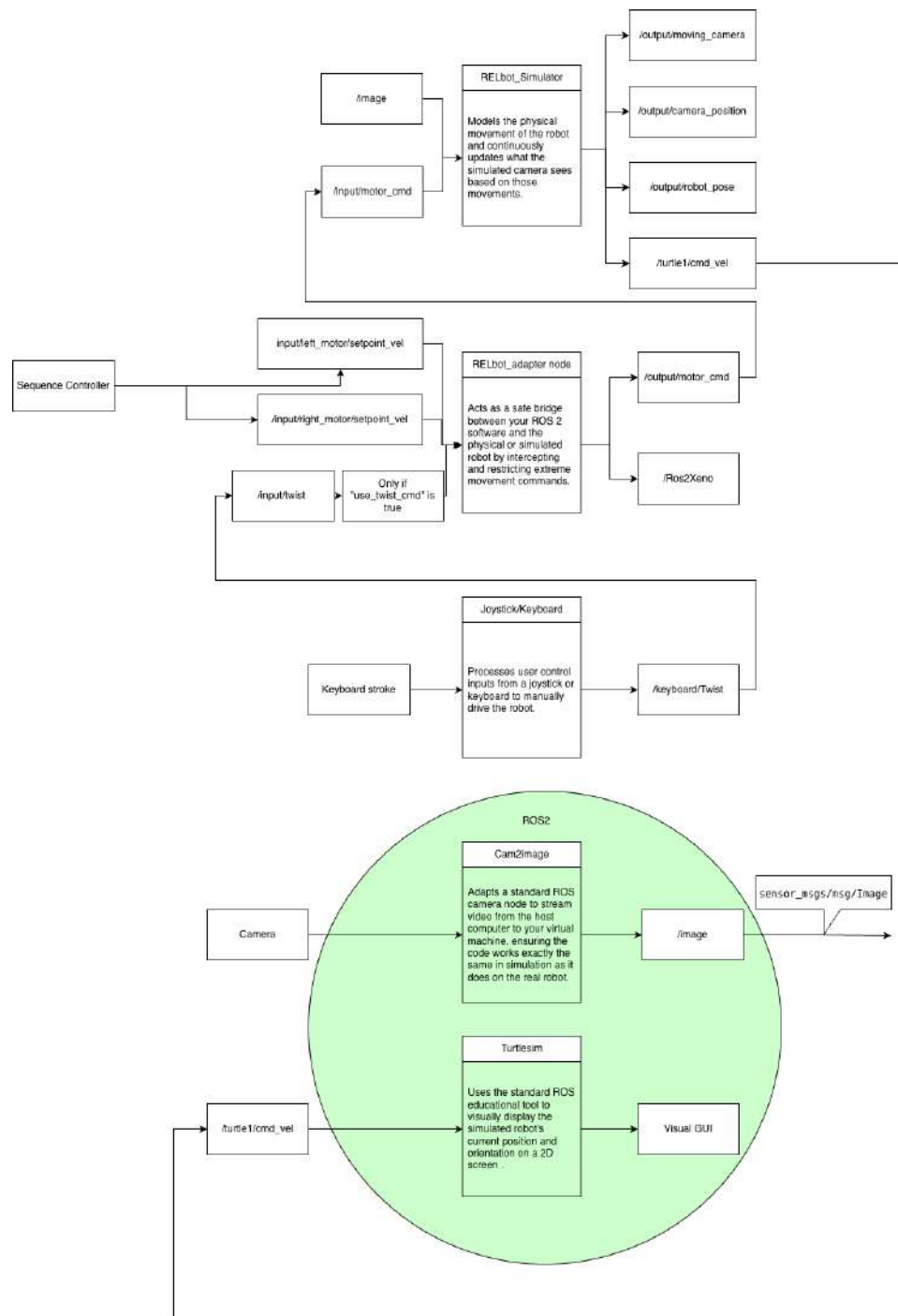
Cam2image node. This node is the visual input interface of the system. It receives the webcam stream from the host computer and publishes it inside ROS 2 on a standard image topic. That makes it possible to test the vision pipeline in simulation using the same kind of interface that is needed later on the physical robot.

RELbot adapter node. The RELbot adapter acts as the routing and safety block between the high-level ROS 2 controller and the plant side of the system. In the current simulation setup it converts the left and right wheel setpoints from the sequence controller into the combined motor command message used by the simulator.

RELbot simulator node. The simulator receives the motor command, updates the robot dynamics, and generates the moving-camera output. This is the most important part of the feedback loop, because a change in robot pose immediately changes the image seen by the detector.

Turtlesim visualisation node. This node provides a simple two-dimensional visualisation of the simulated robot motion. It is not part of the control loop itself, but it is useful for checking whether the simulated pose behaves as expected.

Manual control interfaces. In the broader RELbot architecture, manual input via keyboard or joystick can also exist. Those interfaces are useful for testing, but they were not part of the automated object-tracking setup developed in this report, so they are not discussed further.



Software-oriented block diagram of the RELbot simulation setup. This figure shows the main software blocks around the simulator and adapter. Some paths, such as joystick or

keyboard input, belong to the broader architecture but are not part of the final automated object-tracking experiment.

3 Section 6.2: Detect a Coloured Object

3.1 Section 6.2.1: Testing the Updated `cam2image` Package

The first step was to validate the updated `cam2image_vm2ros` package. Because the Ubuntu development environment runs in a VM and cannot directly access the laptop webcam, the webcam stream was sent to ROS 2 over UDP using the provided `videosever.py` script on the host machine.

The node `cam2image` was configured to run in remote mode with the following settings:

- remote input enabled,
- image size set to 300×200 pixels,
- UDP port set to 9999,
- reliable ROS 2 QoS, and
- camera display disabled inside the ROS node.

The purpose of this test was simple: confirm that the host webcam image could be brought into the Ubuntu ROS 2 workspace and republished correctly on the `/image` topic. This had to work before the image-processing pipeline could be tested.

Design decisions. The camera stream was sent over UDP because that is the workflow prescribed by the assignment manual for VM-based development. The image size was kept moderate so that the stream stayed responsive while still preserving enough detail for ball detection. A launch-file based setup was used so that the experiment could be reproduced easily during grading.

Validation approach. The node was validated by launching the webcam stream and checking whether the image topic could be visualised and whether the downstream simulator received frames consistently.

```

Last login: Mon Apr 13 17:58:26 on ttyS004
user@wlan835816: ~ % cd /Users/user/Desktop/Software_Assignment_6/cam2image_host2vm
python3 videosever.py

0.0.0.0
Listening at: ('0.0.0.0', 9999)
Got connection from ('192.168.2.2', 34825)
Capture dimension set to 380x260
*Traceback (most recent call last):
  File "/Users/user/Desktop/Software_Assignment_6/cam2image_host2vm/videosever.py", line 94, in <module>
    key = cv2.waitKey(1) & 0xFF
KeyboardInterrupt

user@wlan835816: ~ % cd /Users/user/Desktop/Software_Assignment_6/cam2image_host2vm
python3 videosever.py

0.0.0.0
Listening at: ('0.0.0.0', 9999)
Got connection from ('192.168.2.2', 44986)
Capture dimension set to 380x260
[]

user - ubuntu@sdf: ~/ros2_ws - ssh -X ubuntu@sdf - 102x54
ubuntu@sdf:~/ros2_ws$ cd ~/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch cam2image_vm2ros cam2image.launch.py
[INFO] [launch]: All log files can be found below /home/ubuntu/.ros/log/2026-04-17-11-18-28-998011-sdf
x-116483
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [cam2image-1]: process started with pid [115684]
[cam2image-1] [INFO] [1776424709.251756247] [remote_cap]: Remote capture endpoint set to: 192.168.2.1:
9999
[cam2image-1] [INFO] [1776424709.251886914] [remote_cap]: Socket is already open, using this instead
[cam2image-1] [INFO] [1776424709.251886247] [remote_cap]: Sent camera size to endpoint
[cam2image-1] [INFO] [1776424709.82723658] [cam2image]: Publishing image #1
[cam2image-1] [INFO] [1776424719.855995546] [cam2image]: Publishing image #146
[cam2image-1] [INFO] [1776424719.855995546] [cam2image]: Publishing image #296
[cam2image-1] [INFO] [1776424724.858183442] [cam2image]: Publishing image #446
[cam2image-1] [INFO] [1776424729.86188862] [cam2image]: Publishing image #596
[cam2image-1] [INFO] [1776424734.863114916] [cam2image]: Publishing image #746
[cam2image-1] [INFO] [1776424739.924480121] [cam2image]: Publishing image #896
[cam2image-1] [INFO] [1776424744.988248948] [cam2image]: Publishing image #1046
[cam2image-1] [INFO] [1776424749.987331789] [cam2image]: Publishing image #1196
[cam2image-1] [INFO] [1776424754.994657694] [cam2image]: Publishing image #1346
[cam2image-1] [INFO] [1776424768.027986464] [cam2image]: Publishing image #1496
[cam2image-1] [INFO] [1776424765.029124368] [cam2image]: Publishing image #1646
[cam2image-1] [INFO] [1776424776.049710124] [cam2image]: Publishing image #1796
[cam2image-1] [INFO] [1776424775.042762738] [cam2image]: Publishing image #1946
[cam2image-1] [INFO] [1776424788.065335279] [cam2image]: Publishing image #2096
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #2246
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #2396
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #2546
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #2696
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #2846
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #2996
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #3146
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #3296
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #3446
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #3596
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #3746
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #3896
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #4046
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #4196
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #4346
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #4496
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #4646
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #4796
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #4946
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #5096
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #5246
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #5396
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #5546
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #5696
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #5846
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #5996
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #6146
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #6296
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #6446
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #6596
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #6746
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #6896
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #7046
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #7196
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #7346
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #7496
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #7646
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #7796
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #7946
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #8096
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #8246
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #8396
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #8546
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #8696
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #8846
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #8996
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #9146
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #9296
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #9446
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #9596
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #9746
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #9896
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #10046
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #10196
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #10346
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #10496
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #10646
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #10796
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #10946
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #11096
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #11246
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #11396
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #11546
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #11696
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #11846
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #11996
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #12146
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #12296
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #12446
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #12596
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #12746
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #12896
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #13046
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #13196
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #13346
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #13496
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #13646
[cam2image-1] [INFO] [1776424785.047783911] [cam2image]: Publishing image #13796

```

Figure 3: Cam2image node launch

Result of testing the updated cam2image node.

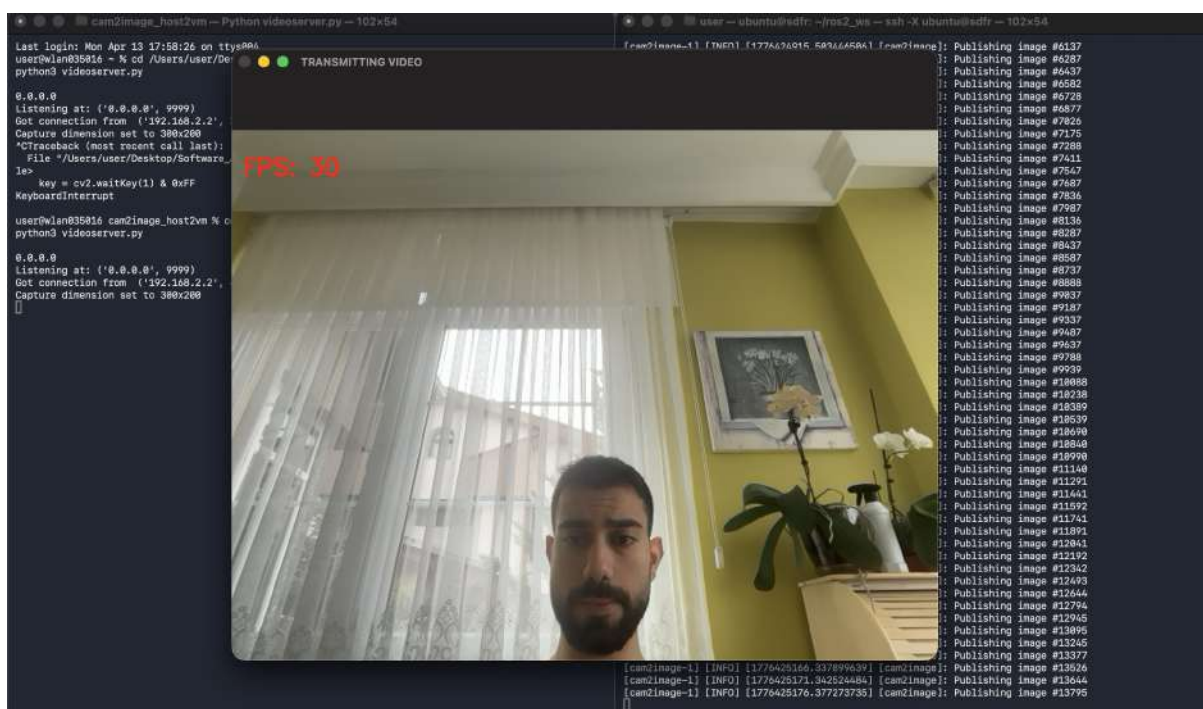


Figure 4: Camera feed

3.2 Section 6.2.2: Image Processing — Detecting the Green Object

After validating the image input pipeline, a dedicated ROS 2 node named `ball_detector` was implemented to detect a green object in the moving camera image produced by the simulator.

Node interfaces. The implemented detector subscribes to:

- `/output/moving_camera` (`sensor_msgs/Image`)

and publishes:

- `/ball_position` (`geometry_msgs/Point`)

The meaning of the published point is:

- x : horizontal image position of the ball centroid,
- y : vertical image position of the ball centroid,
- z : detected blob area in pixels.

Detection method. The incoming image is first converted from ROS image format to an OpenCV matrix. After that, the image is transformed from BGR colour space to HSV colour space. HSV was chosen because it is more robust for colour thresholding than thresholding directly in BGR.

Green pixels are selected using fixed thresholds:

- hue: 35 to 85,
- saturation: 70 to 255,
- value: 70 to 255.

After thresholding, morphological opening and closing with a 5×5 elliptical kernel are applied to suppress noise and fill small gaps in the mask. The centroid and area of the remaining blob are then computed using image moments. Small detections with an area below 500 pixels are rejected. For valid detections, the centroid and blob area are published.

Design decisions. The node was kept intentionally simple and reusable:

- The detector publishes a generic `geometry_msgs/Point` instead of a specialised message type.
- The image topic is configurable via a ROS parameter, making the node reusable with either the raw camera stream or the simulator moving-camera output.
- Blob area is used as a simple measure of apparent object size, which is useful later for forward/backward control.

Validation approach. The detector was tested by moving a green object in front of the webcam and observing whether the reported position changed consistently with the visible object. The node output was also connected to the control setup so that its behaviour could be checked inside the full closed-loop simulation.

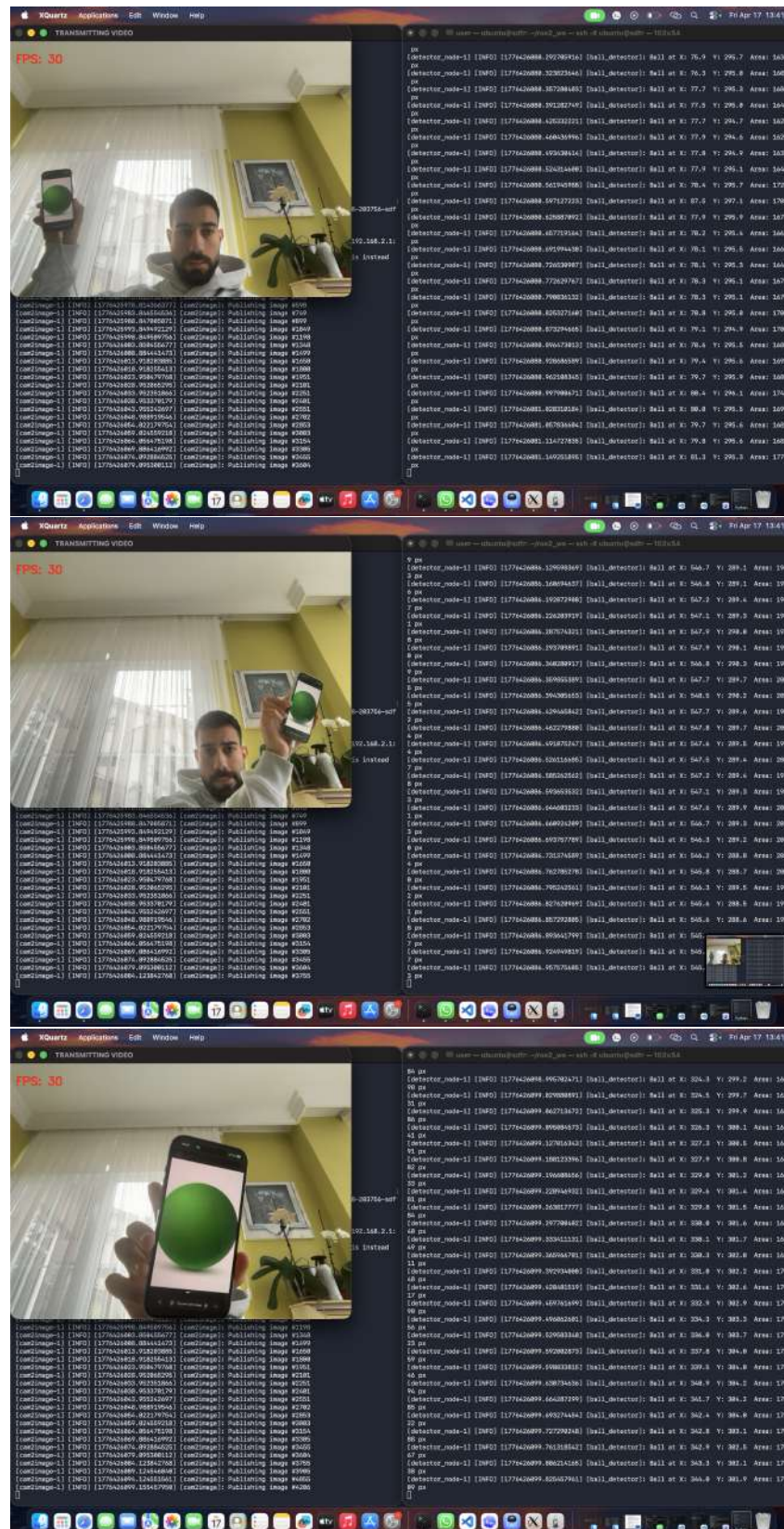


Figure 5: Ball detector node outputs

4 Section 6.3: Sequence Controller

The next step was to extend the sequence controller so that the simulated RELbot follows the detected green object. The controller uses the output of the detector node and drives the simulated RELbot so that the object moves towards the centre of the camera view.

4.1 Relation to the earlier sequence controller

The controller developed in Assignment 5.3 served as the starting point for this work. In that earlier version, the sequence controller published wheel velocity setpoints at 30 Hz and used elapsed time to switch between motion phases. With that open-loop approach, the simulated robot could execute recognisable paths such as a straight line, a circle, an L-shape, and a square with rounded corners.

That earlier work also made it clear why feedback is needed in Assignment 6. The realised paths deviated from the ideal mathematical shapes for two main reasons. First, the simulator models the motors as a first-order system with a time constant of 0.3 s, so wheel speeds change gradually rather than instantly. Second, a purely time-based controller cannot compensate for drift or disturbances because it does not measure the actual result of its commands. For that reason, the Assignment 5 controller was extended into a visual closed-loop controller.

The physical constants used earlier remained relevant here as well, especially the robot width $W = 0.209$ m and the wheel radius $r = 0.05$ m. These values provide the kinematic basis for translating high-level control behaviour into left and right wheel commands.

4.2 Control objective

The assignment manual suggests a first-order controller in which the robot pose is driven towards the object position. In the implemented setup, the detector provides the object position directly in image coordinates, while the simulator provides the moving-camera image that reflects the current robot pose. For that reason, the controller was implemented as an image-based feedback controller that uses:

- horizontal image offset for steering,
- detected blob area as a proxy for object distance,
- image width from the camera topic to compute the image centre,
- a search behaviour when the object is temporarily lost.

4.3 Implemented node interfaces

The node `relbot_sequence_controller` subscribes to:

- `/ball_position` (`geometry_msgs/Point`),
- `/output/moving_camera` (`sensor_msgs/Image`).

It publishes:

- `/input/left_motor/setpoint_vel` (example_interfaces/Float64),
- `/input/right_motor/setpoint_vel` (example_interfaces/Float64).

These wheel setpoints are then converted by the `relbot_adapter` node into `/output/motor_cmd`, which is consumed by the simulator.

4.4 Implemented control law

The controller runs at 30 Hz. Let x_b be the detected horizontal ball position, w the current image width, and A_b the detected blob area. The image centre is taken as $w/2$. A normalised steering error is computed as:

$$e_\theta = \frac{x_b - w/2}{w/2}.$$

This value is clamped to the range $[-1, 1]$. A small deadband of 4 pixels is used around the image centre to avoid unnecessary oscillation.

The forward motion is based on the apparent blob size. A target area A_{target} is chosen, and the normalised area ratio is:

$$r_A = \text{clamp}\left(\frac{A_b}{A_{\text{target}}}, 0, 2\right).$$

When $r_A \geq 1$, the object is considered close enough and the robot stops. Otherwise, the forward velocity is reduced as the object gets closer:

$$v = K_V(1 - r_A),$$

with clamping between a minimum and maximum value.

The steering command is computed as:

$$\omega = K_{\text{steer}} e_\theta,$$

again with saturation. While turning strongly, the forward speed is reduced to obtain smoother behaviour. The wheel setpoints are then formed from the forward and steering terms and published to the simulator chain.

Search behaviour. If no valid object is received for longer than the configured timeout, the controller switches to a search mode. In this mode, the robot rotates in place until the ball is detected again. The search direction is based on the last known side of the object in the image, which makes reacquisition faster than always turning in the same direction.

Parameter values. The final controller was tuned with the following main values:

- loop rate: 30 Hz,
- target blob area: 9000 pixels,
- centre deadband: 4 pixels,
- ball timeout: 0.35 s,
- search speed: 0.55,
- maximum wheel command: 1.50.

4.5 System integration

The complete simulation setup was launched with a single launch file. In this setup:

- `cam2image` publishes the incoming webcam image on `/image`,
- `relbot_simulator` generates `/output/moving_camera`,
- `ball_detector` processes `/output/moving_camera`,
- `relbot_sequence_controller` computes wheel commands from the detector output,
- `relbot_adapter` converts wheel commands to simulator motor commands,
- `relbot2turtlesim` visualises the simulated pose in turtlesim.

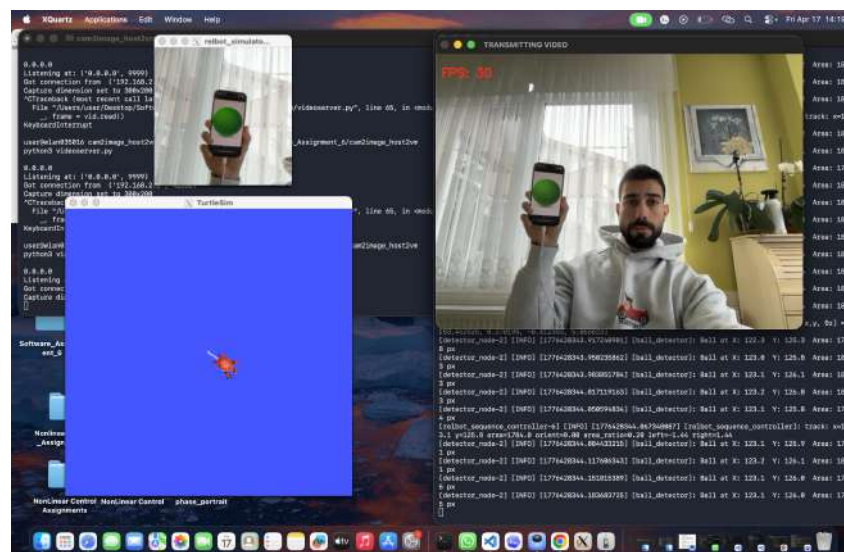
This structure keeps the system modular. The detector is separate from the controller, and the controller only assumes that some perception node publishes object position and size on a topic.

4.6 Validation approach

The controller was validated in simulation by moving the green object in front of the webcam and checking whether:

- the simulated robot rotated to centre the object,
- the robot moved forward when the object was far away,
- the robot stopped when the object became sufficiently large in view,
- the robot searched for the object when the object disappeared from view.

The turtlesim visualisation was used as an extra check that the robot pose changed consistently with the control objective.



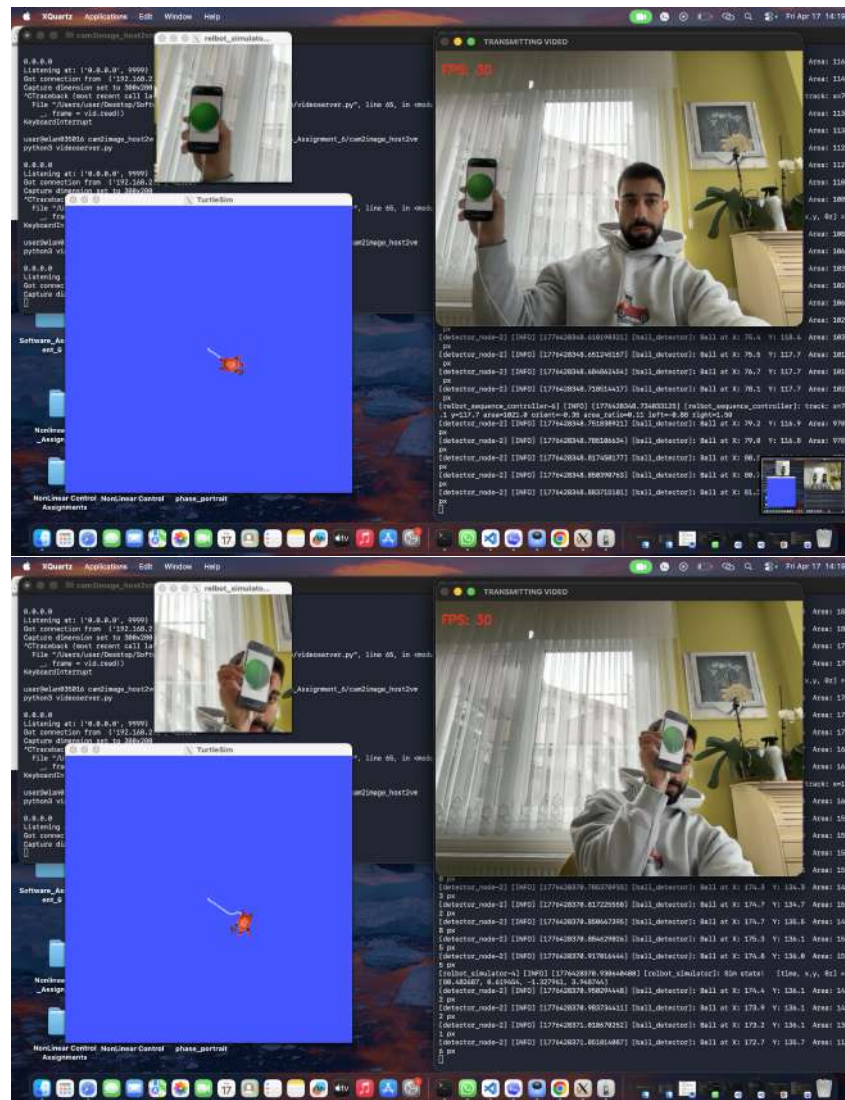
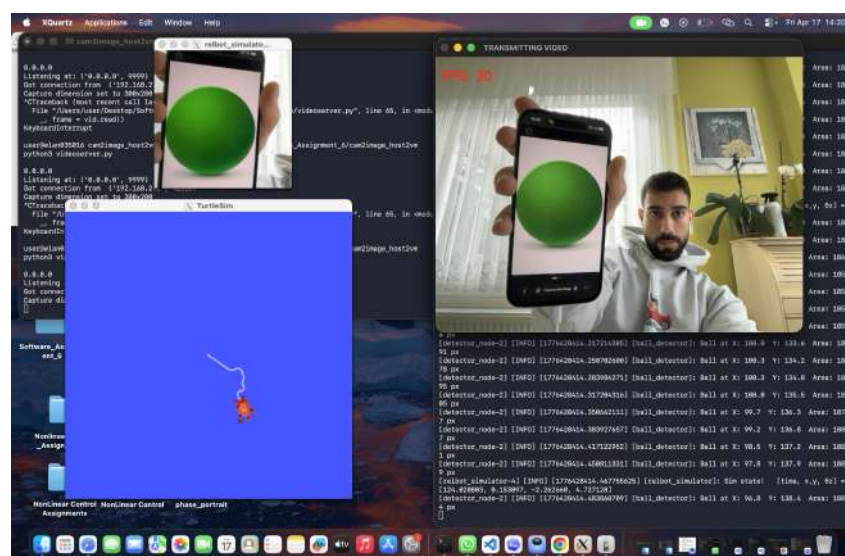


Figure 6: Turtlesim going forward/right/left towards ball



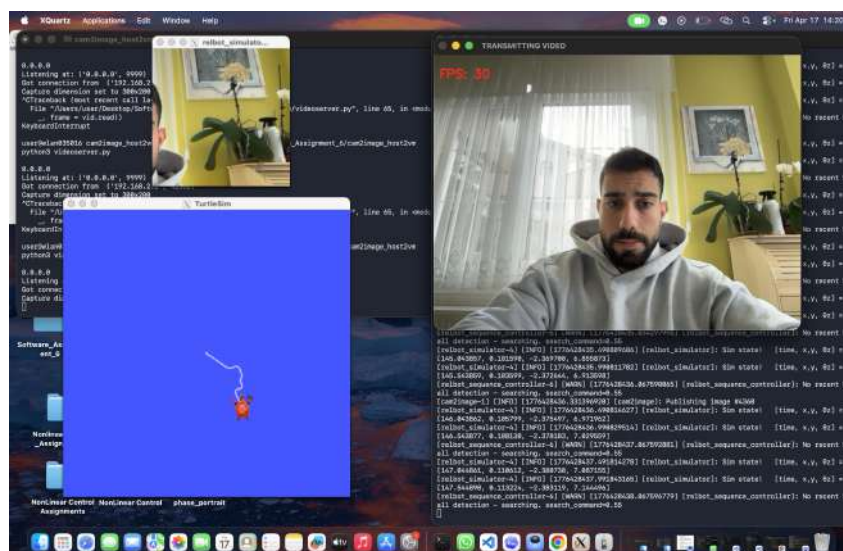


Figure 7: Turtlesim stopping when the ball is too close and rotating around itself when the ball is lost

5 Discussion

The developed system matches the intended workflow of Assignment 6 up to Section 6.3: a webcam image is brought into ROS 2, processed to detect a green object, and then used in a closed-loop simulation to steer the RELbot towards that object. The most important design choice was to let the detector work on the simulator moving-camera output rather than on the raw webcam stream. Because of that, the robot motion changes the camera view, and the changed camera view affects the next control action. This creates the closed loop that the assignment is aiming for.

Another important design choice was to keep the system modular. The detector, controller, adapter, simulator, and turtlesim visualisation all remain separate nodes with clear interfaces. That makes the setup easier to test, easier to explain, and easier to migrate later to the real robot.

6 Section 6.4: Controlling the Physical RELbot

6.1 Section 6.4.1: Controller-Code Stack on the Raspberry Pi

The first step was to migrate the complete controller-code stack developed and validated in Section 6.3 to the Raspberry Pi embedded in the RELbot. The goal at this stage was not yet to drive the motors, but to run the same ROS 2 nodes on the Pi together with the RELbot simulator, verifying that the code executes correctly on the ARM-based processor architecture.

The following packages were built on the Raspberry Pi using `colcon`:

- **ball_detector** — detects the coloured ball in the camera image and publishes its centroid and area,
- **relbot_sequence_controller** — computes wheel velocity commands from the ball position,

- `relbot_launch` — launch files for both the simulation and the hardware setup.

After building, the simulation was launched directly on the Raspberry Pi:

```
ros2 launch relbot_launch full_tracking_system.launch.py
```

All nodes started and communicated correctly. The ball-tracking behaviour was functionally equivalent to the laptop execution. The `showimage` display was accessed remotely via X forwarding (`ssh -X`), which confirmed that visual output was available without a local graphical environment on the Pi.

Remark on the hardware launch file. The combined launch file `lab.hardware.launch.py` did not function correctly on the Raspberry Pi during the lab session. The suspected cause is a combination of environment sourcing differences and node startup ordering that the launch file does not explicitly enforce. As a workaround, all nodes were started individually across separate terminals, which allowed full operation of the real RELbot. This issue is discussed further in Section 6.5.2.

6.2 Section 6.4.2: Basic Test of the Provided Loop Controller

Before combining the full controller stack with the FRT loop controller, a set of basic motor tests was performed to verify that the loop controller responds correctly to explicit setpoints. During all tests the RELbot was left on its stand to prevent uncontrolled movement.

System startup. The FRT loop controller (`RelBotDemo / LoopController`) was started via the Xenomai XRF2 framework on the FRT core of the Raspberry Pi:

```
sudo ./build/demo/demo
```

The ROS–Xenomai bridge and the RELbot adapter were then launched:

```
ros2 run ros_xeno_bridge RosXenoBridge
```

```
ros2 run relbot_adapter relbot_adapter --ros-args \
  -p robotmode:="real" -p use_twist_cmd:=false \
  -p output_rate_hz:=50.0 -p command_timeout_sec:=0.30 \
  -p max_speed_mps:=0.25 -p max_speed_rads:=5.0
```

The controller was then initialised and started by publishing to the `/XenoCmd` topic:

```
ros2 topic pub --once /XenoCmd std_msgs/msg/Int32 "{data: 1}"
ros2 topic pub --once /XenoCmd std_msgs/msg/Int32 "{data: 2}"
```

Test cases. Three basic motion tests were carried out with the RELbot on its stand:

1. **Left motor only** — a positive velocity setpoint was sent to the left motor while the right motor setpoint was zero. The left wheel rotated as expected; the right wheel remained stationary.
2. **Right motor only** — a symmetric test for the right motor produced the same result on the opposite side.

3. **Both motors** — equal setpoints were sent to both motors simultaneously. Both wheels spun at equal speed, consistent with straight-line driving. Differential setpoints were also tested to verify turning behaviour.

Evaluation. All tests produced the expected wheel behaviour. The loop controller responded promptly to incoming setpoints, and the command-timeout mechanism correctly stopped the motors when no new command was received within the configured 0.30 s window. This confirmed that the loop controller and adapter chain behave as expected before the full vision-based stack is connected.

<https://youtu.be/6k946abHX0k>

6.3 Section 6.4.3: Questions

Question 1: Does the simulated system perform on the Raspberry Pi as well as on your own laptop? Why (not)? The simulated system ran correctly on the Raspberry Pi and the visual output via X forwarding matched the behaviour observed on the laptop. However, the Raspberry Pi has a significantly lower clock frequency and a different CPU architecture (ARM Cortex-A72) compared to a modern x86-64 laptop, which means it has less raw computational throughput. As a result, image processing in the `ball_detector` node introduced slightly more latency, and overall CPU utilisation was noticeably higher than on the laptop.

Nevertheless, all ROS 2 nodes ran within their timing requirements and the overall tracking behaviour remained functionally equivalent. The primary conclusion is that the Raspberry Pi is resource-constrained but adequate for this controller stack.

Question 2: Did you have to make changes to the controller-code stack while migrating it from your laptop to the Raspberry Pi? If so, why was it necessary? Could you have avoided this in the previous subassignments? The Python source code of the ROS 2 nodes required no changes. ROS 2 and Python are cross-platform by design, and the ARM build produced functionally equivalent executables. The issue that did arise was with the launch file `lab_hardware.launch.py`: it failed to bring up all nodes correctly on the Pi environment, most likely due to differences in environment variable sourcing and node startup ordering between the Pi's ROS 2 installation and the laptop setup.

This issue could have been partially avoided by testing the hardware launch file against the Raspberry Pi earlier, ideally already at the start of Section 6.4.1. More robust launch file design, for example, by adding explicit `ros2_launch` delay actions between nodes or by ensuring all environment variables are sourced explicitly rather than relying on a pre-sourced shell would likely have prevented the problem.

Question 3: Did the setpoints you provided in Section 6.4.2 match with those you made in your controller stack? If not, what did you need to change? The setpoints used in the basic motor test were consistent with the limits configured in the sequence controller during simulation. The adapter was configured with a maximum linear speed of 0.25 m/s and a maximum rotational speed of 5.0 rad/s, which matched the operating range used during simulation. No parameter changes were necessary at this stage.

The one setpoint that could not be fully validated during the stand test was the target blob area (9000 px²), which controls the desired apparent size of the ball and therefore the forward/backward distance the RELbot maintains. Since the wheels spin freely on the stand, the robot cannot actually approach or retreat from the ball. This parameter was instead validated during the final integration in Section 6.5.

7 Section 6.5: Final Integration

7.1 Section 6.5.1: Integrate All and Final Demonstration

With the basic motor tests in Section 6.4.2 confirming that the loop controller and adapter chain operate correctly, the complete system was assembled by combining the controller-code stack with the FRT loop controller running on the Xenomai core of the Raspberry Pi. The goal was to let the RELbot follow the coloured ball on the floor of the lab.

System startup procedure. Because the combined launch file did not work on the Raspberry Pi (see Section 6.4.1), all components were started individually across six terminals:

- **Terminal 1** (FRT loop controller, outside ROS): `sudo ./build/demo/demo`
- **Terminal 2** (RELbot adapter):
`ros2 run relbot_adapter relbot_adapter --ros-args`
`-p robotmode:="real" -p use_twist_cmd:=false`
`-p output_rate_hz:=50.0 -p command_timeout_sec:=0.30`
`-p max_speed_mps:=0.25 -p max_speed_rads:=5.0`
- **Terminal 3** (ROS–Xenomai bridge): `ros2 run ros_xeno_bridge RosXenoBridge`
- **Terminal 4** (camera): `ros2 run image_tools cam2image`
- **Terminal 5** (ball detector):
`ros2 run ball_detector detector_node --ros-args -p image_topic:="/image"`
- **Terminal 6** (sequence controller):
`ros2 run relbot_sequence_controller relbot_sequence_controller --ros-args`
`-p image_topic:="/image" -p target_ball_area:=9000.0`
`-p center_deadband_px:=4.0 -p ball_timeout:=0.35`

After all nodes were running, the controller was initialised and started:

```
ros2 topic pub --once /XenoCmd std_msgs/msg/Int32 "{data: 1}"
ros2 topic pub --once /XenoCmd std_msgs/msg/Int32 "{data: 2}"
```

Demonstration results. The RELbot was placed on the floor and the coloured ball was introduced into the camera’s field of view. The following behaviours were observed:

- **Steering:** when the ball was moved to the left or right of the image centre, the RELbot rotated in the corresponding direction to re-centre the ball in view.

- **Forward approach:** when the ball was far from the camera, the robot drove forward. As the ball grew larger in the image, forward speed reduced proportionally. The robot stopped when the blob area reached the target of 9000 px².
- **Ball loss:** when the ball was moved outside the field of view, the robot stopped within the 0.35 s timeout and then rotated in place to search for the ball, consistent with the search behaviour implemented in the sequence controller.

The system behaved as expected. The ball was moved slowly and the RELbot followed accordingly, consistent with the simulation results from Section 6.3. A short video of the RELbot following the coloured ball was recorded during the lab session as evidence.

https://youtube.com/shorts/wcJdy_U8lG8

7.2 Section 6.5.2: Questions

Question 1: Did the final integration work immediately (*first-time right*)? If not, what did you need to change? In hindsight, how could you have avoided the need to make changes in this last integration phase? The final integration did not work entirely first-time right. The primary issue was the `lab_hardware.launch.py` launch file, which was intended to start all hardware-facing nodes in a single command but failed on the Raspberry Pi environment. This forced a manual multi-terminal startup procedure, which added overhead and increased the risk of startup errors during the lab session.

Once all nodes were started manually in the correct order, the ball-tracking behaviour worked correctly with only minor observation. The controller parameters tuned in simulation (target area, deadband, timeout, and speed limits) transferred well to the real system without requiring changes. This is an encouraging result it confirmed that the simulation was a faithful enough representation of the real RELbot for controller design purposes.

While the steering logic (ω) worked perfectly in the idealized simulation, the physical RELbot exhibited an inverted response. This divergence was caused by the inherent electrical polarity of the physical motor drives and how the low-level Xenomai loop translates velocity commands. Consequently, the mathematically “correct” commands caused the robot to turn away from the target, requiring us to explicitly invert the ω values in software to synchronize with the real-world hardware.

In hindsight, the launch file failure could have been avoided by testing `lab_hardware.launch.py` on the Raspberry Pi already at the start of Section 6.4.1, as part of the migration step. Testing the deployment environment early rather than only at the moment of final integration is the key lesson here. More generally, the mismatch between simulation and reality was small in this case, but the integration phase is always where such differences become visible. Building and testing each layer incrementally on the target hardware, rather than deferring all hardware testing to the final step, would make future integrations more robust.

A Apendix 1 - AI statement

During the preparation of this work, the authors used, Claude.ai, and Overleaf Write-full to rewrite sentences and for the code development. After using this tool/service, the authors reviewed and edited the content as needed and took full responsibility for the content of the work.